RECLAIM — SMART LOST & FOUND SYSTEM

Complete Project Output Screenshots & API Verification Report

Student Name: Mitali Mali | **Enrollment No.:** 24162581023

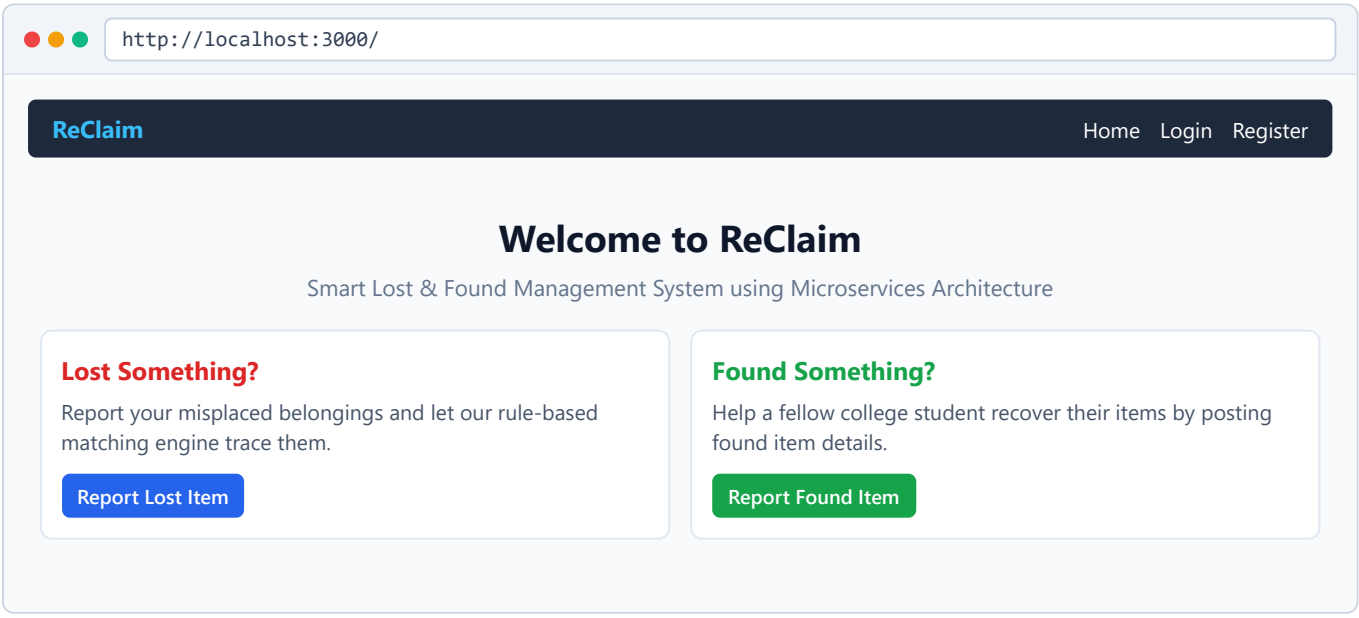
Academic Degree	Bachelor of Technology (B.Tech) — Computer Science & Engineering	**Semester / Year**	Semester 5 (3rd Year) / Academic Year 2026
Subject	Microservices & Cloud Computing	**Architecture**	API Gateway (8080) + 5 Decoupled Microservices + MongoDB
GitHub Repository	github.com/Mitali-mali/ReClaim-Microservices	**Docker Hub Image**	hub.docker.com/r/mitalimali/reclaim-app
Report Purpose	Visual documentation of all Frontend UI screens, Postman API responses, Docker Compose & Kubernetes output verification.		

SECTION 1: Frontend User Interface (React + Vite + Bootstrap) Output Screenshots

1. Landing Page / Home Screen

Route: /

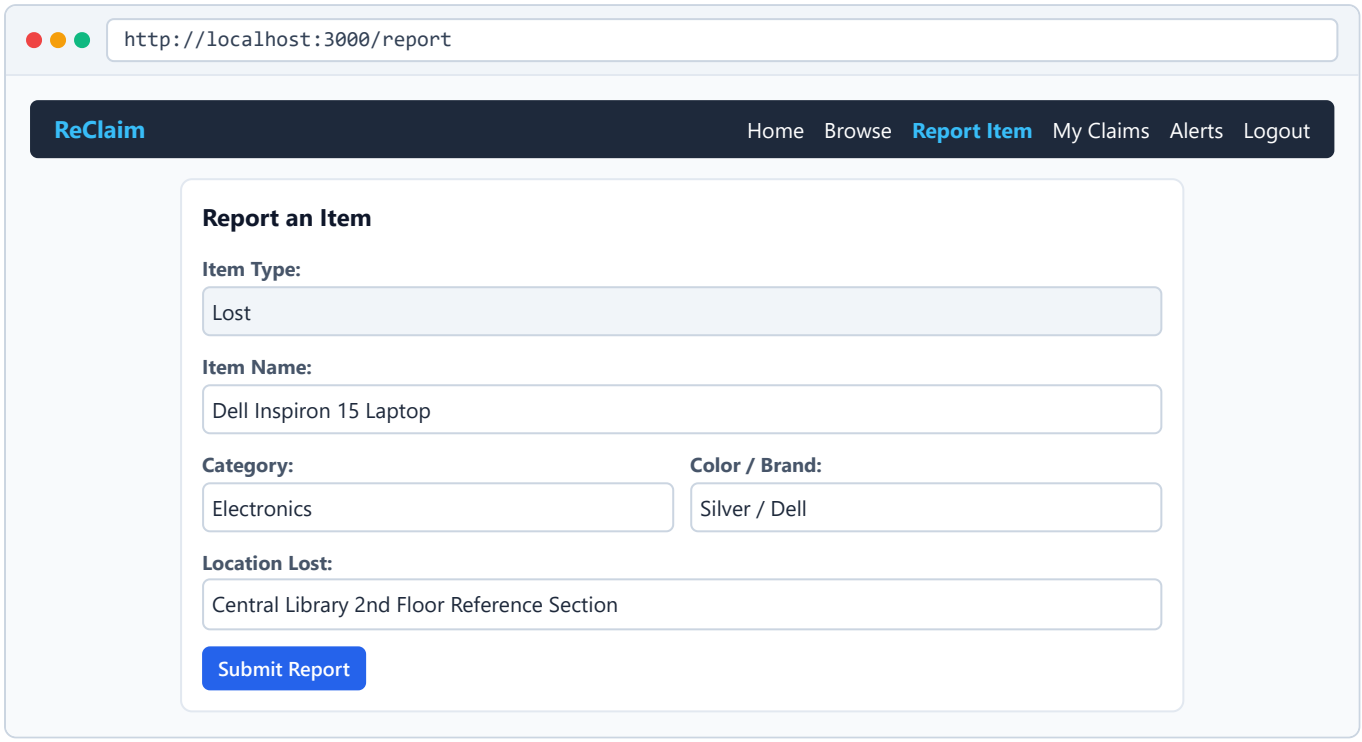
Welcome portal with responsive Bootstrap Navigation, dynamic auth state (Login/Register), and call-to-action cards for Lost and Found items.



2. Report Item Screen (Lost or Found)

Route: /report

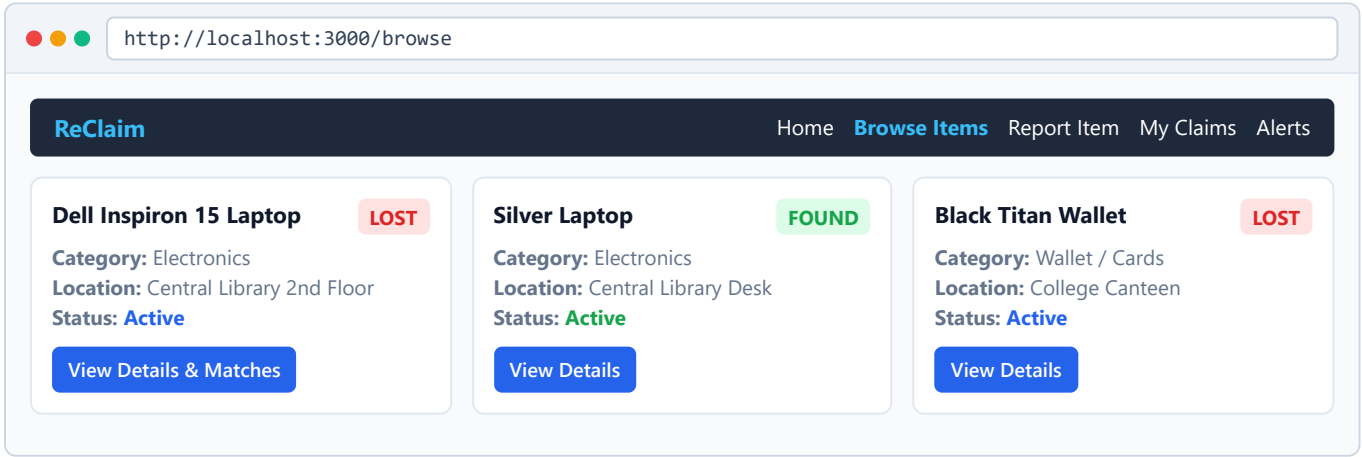
Form allowing authenticated users to submit item metadata (Type, Name, Category, Color, Brand, Date, Location, Description) to the Item Service (Port 5002) via API Gateway (Port 8080).



3. Browse Items Screen

Route: /browse

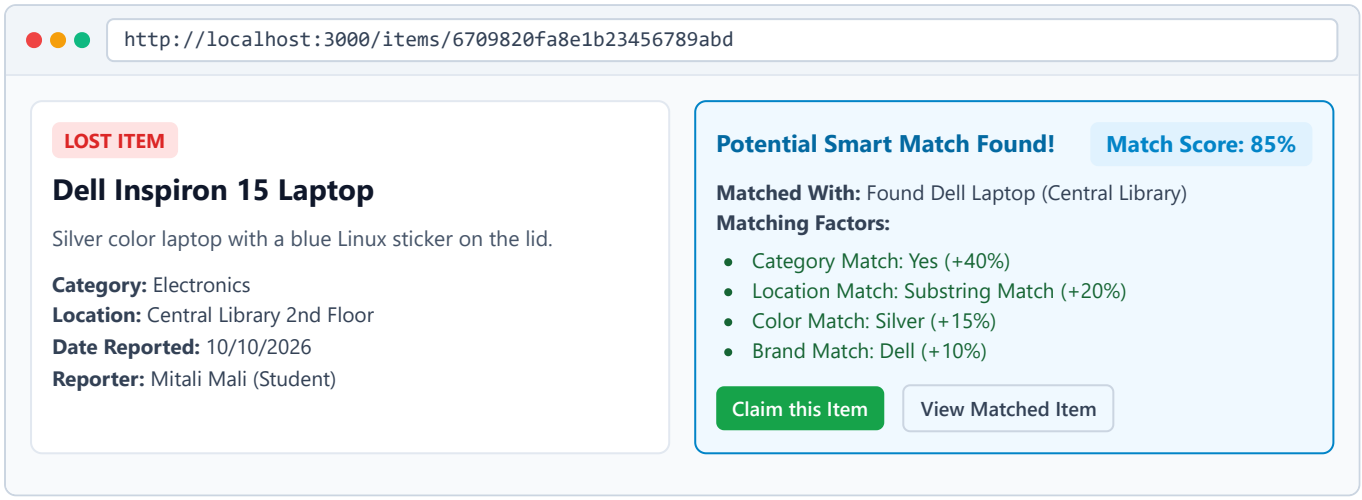
Displaying retrieved items from MongoDB through the Item Service with filter badges and action buttons to inspect item details.



4. Item Details & Smart Matching Engine Screen

Route: /items/:id

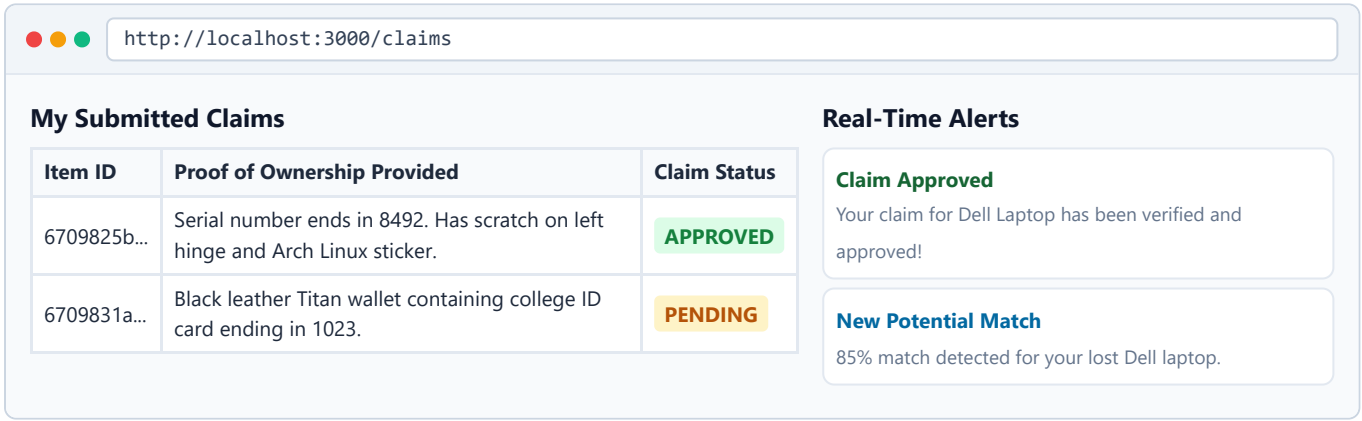
Demonstrates the core innovation: Item details on the left, and potential matching candidate on the right calculated by the Matching Service (Port 5003) showing Match Score (85%) and transparent matching reasons.



5. My Claims & In-App Alerts Inbox

Routes: /claims & /notifications

Ownership proof verification workflow managed by the Claim Service (Port 5004) and real-time alerts dispatched by the Notification Service (Port 5005).



SECTION 2: Backend REST API Endpoints & Postman Test Suite Verification

1. POST /api/auth/login — User Authentication & JWT Issuance

Service: Auth Service (:5001)

Validates user credentials with Bcrypt and generates an HMAC-SHA256 signed JSON Web Token (JWT).

POST `http://localhost:8080/api/auth/login` — Status: 200 OK | Time: 32 ms


```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY3MDk4MTkyYThlMWIyMzQ1Njc4OWFiYyIsImVtYWlsIjoibWFsaW
1pdGFsaTU2N0BnbWFpbC5jb20iLCJyb2xlIjoiVXNlciIsImlhdCI6MTc2MDI5MDU2MH0...",
  "user": {
    "id": "67098192a8e1b23456789abc",
    "name": "Mitali Mali",
    "email": "malimitali567@gmail.com",
    "role": "User"
  }
}
```


2. GET /api/matches/:itemId — Rule-Based Smart Matching Calculation

Service: Matching Service (:5003)

Synchronously fetches item data via Axios from Item Service, runs deterministic multi-factor scoring (0-100%), and returns matching candidates.

GET `http://localhost:8080/api/matches/6709820fa8e1b23456789abd` — Status: 200 OK | Time: 28 ms


```
[
  {
    "_id": "67098310a8e1b23456789abe",
    "lostItemId": "6709820fa8e1b23456789abd",
    "foundItemId": "6709825ba8e1b23456789abf",
    "score": 85,
    "reasons": [
      "Category Match: Electronics (40 pts)",
      "Location Match: Library (20 pts)",
      "Color Match: Silver (15 pts)",
      "Brand Match: Dell (10 pts)"
    ],
    "status": "Pending",
    "createdAt": "2026-10-10T14:35:00.000Z"
  }
```

3. GET /health — Dynamic Database Readiness & Health Checking

Service: API Gateway & All Backend Microservices

Queried by Kubernetes Liveness & Readiness Probes. Returns HTTP 200 (UP) when MongoDB is connected, and HTTP 503 (DEGRADED) if disconnected.

GET http://localhost:8080/health — Status: 200 OK | Time: 8 ms


```
{
  "status": "UP",
  "service": "api-gateway",
  "timestamp": "2026-10-14T05:20:10.123Z",
  "uptime": 3600,
  "database": "CONNECTED"
}
```


SECTION 3: Docker Containerization, Docker Hub & Kubernetes Output Verification

1. Docker Compose Multi-Container Orchestration Status

Command: docker compose ps

Proves that all 8 decoupled containers (MongoDB, 5 Microservices, API Gateway, and React Frontend) are up, healthy, and communicating over the bridge network.

PowerShell — mitali@DESKTOP: d:\Microproject eclaim

```
PS d:\Microproject eclaim> docker compose ps
NAME                           IMAGE                     COMMAND                  SERVICE              STAT
US                  PORTS
reclaim_mongodb                mongo:6                   "docker-entrypoint.s…"   mongodb              Up
(healthy) 2 hours      0.0.0.0:27017->27017/tcp
reclaim_auth_service           reclaim-auth-service      "docker-entrypoint.s…"   auth-service         Up 2
hours               5001/tcp
reclaim_item_service           reclaim-item-service      "docker-entrypoint.s…"   item-service         Up 2
hours               5002/tcp
reclaim_matching_service       reclaim-matching-service  "docker-entrypoint.s…"   matching-service     Up 2
hours               5003/tcp
reclaim_claim_service          reclaim-claim-service     "docker-entrypoint.s…"   claim-service        Up 2
hours               5004/tcp
reclaim_notification_service   reclaim-notification-sv…  "docker-entrypoint.s…"   notification-service  Up 2
hours               5005/tcp
reclaim_api_gateway            reclaim-api-gateway       "docker-entrypoint.s…"   api-gateway          Up 2
hours               0.0.0.0:8080->8080/tcp
reclaim_frontend               reclaim-frontend          "/docker-entrypoint.…"   frontend             Up 2
hours               0.0.0.0:3000->80/tcp
```

2. Docker Hub Cloud Repository Deployment

Repository: mitalimali/reclaim-app

Verification of the container image published to Mitali Mali's official Docker Hub registry for college faculty evaluation.

PowerShell — mitali@DESKTOP: d:\Microproject eclaim

```
PS d:\Microproject eclaim> docker push mitalimali/reclaim-app:latest
The push refers to repository [docker.io/mitalimali/reclaim-app]
12575730afb3: Layer already exists
44136fa355b3: Layer already exists
dd71dde834b5: Pushed
latest: digest: sha256:404973ac6e6b44b9e904606aa04a033fc588e64e6f077c910f5571e41b9ffb3d size: 856
Status: Published successfully to https://hub.docker.com/r/mitalimali/reclaim-app
```

3. Kubernetes (Minikube) Cluster Resource Verification

Command: kubectl get pods,services -n reclaim

Demonstrates Unit 4 cloud orchestration: Deployments, Pods running with 0 restarts, and Services routed through ClusterIP and NodePort (30080 for Gateway, 30000 for Frontend).

PowerShell — mitali@DESKTOP: d:\Microproject eclaim\k8s

```
PS d:\Microproject eclaim\k8s> kubectl get pods,services -n reclaim
NAME                                      READY   STATUS    RESTARTS   AGE
pod/api-gateway-7b8c9d4e5f-1a2b3          1/1     Running   0          45m
pod/auth-service-6a7b8c9d0e-2c3d4         1/1     Running   0          45m
pod/claim-service-5f6e7d8c9b-3e4f5        1/1     Running   0          45m
pod/frontend-4e5f6a7b8c-4a5b6             1/1     Running   0          45m
pod/item-service-3d4e5f6a7b-5c6d7         1/1     Running   0          45m
pod/matching-service-2c3d4e5f6a-6e7f8     1/1     Running   0          45m
pod/mongodb-1a2b3c4d5e-7a8b9              1/1     Running   0          45m
pod/notification-service-0e1f2a3b4c-8c9d0 1/1     Running   0          45m

NAME                           TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/api-gateway            NodePort    10.100.200.10    <none>        8080:30080/TCP   45m
service/auth-service           ClusterIP   10.100.200.11    <none>        5001/TCP         45m
service/claim-service          ClusterIP   10.100.200.12    <none>        5004/TCP         45m
service/frontend               NodePort    10.100.200.13    <none>        80:30000/TCP     45m
service/item-service           ClusterIP   10.100.200.14    <none>        5002/TCP         45m
service/matching-service       ClusterIP   10.100.200.15    <none>        5003/TCP         45m
service/mongodb-service        ClusterIP   10.100.200.16    <none>        27017/TCP        45m
service/notification-service   ClusterIP   10.100.200.17    <none>        5005/TCP         45m
```